

인덱스 - index.html 이랑 연관이 있을까

인덱스 개념을 파악하기

데이터베이스에서 데이터를 빠르게 찾을 수 있도록 도와주는 인덱스에 대해 공부한 내용을 정리했다.

0. 시작하기 전

인덱스는 데이터를 빠르게 찾을 수 있도록 도와주는 도구로, 실무에서는 현실적으로 인덱스 없이 데이터베이스 운영이 불가능하다.

인덱스에는 **클러스터형 인덱스**와 **보조 인덱스**가 있다.

클러스터형 인덱스 (Clustered Index)

클러스터형 인덱스는 기본 키로 지정하면 자동 생성되며 테이블에 1개만 만들 수 있다.

기본 키로 지정한 열을 기준으로 **자동 정렬**이 된다.

- 테이블의 실제 데이터 자체가 인덱스 순서대로 정렬되어 저장
- 보통 PK로 자동 생성
- 데이터 자체가 정렬되어 있어 검색 속도가 높으며 테이블당 1개이다

보조 인덱스 (Secondary Index)

보조 인덱스는 고유 키로 지정하면 자동 생성되며 여러 개를 만들 수도 있지만 자동 정렬되지는 않는다.

- 실제 데이터와 별개로 검색용 목차만 따로 관리
- UNIQUE, INDEX, KEY 등으로 수동 생성
- 보조 인덱스 → 클러스터 인덱스 → 실제 데이터 순서이다

1. 인덱스 개념

"데이터베이스 이론"이라는 1000페이지 책이 있다고 가정해보자.

책의 내용 중 UNIQUE 에 대해서 찾아보고 싶다면 어떻게 해야 할까?

두 가지 방법

방법 1: 처음부터 찾기

- 책의 첫 페이지부터 찾아보는 방법

방법 2: 찾아보기 사용

- 책의 제일 뒤에 수록되어 있는 "찾아보기"를 열어서 찾는 방법
- "찾아보기"는 이미 정렬되어 있어 'U' 부분을 살펴보면 쉽게 UNIQUE 단어를 찾을 수 있다
- 단어 옆에 페이지 번호도 적혀 있어 빠르게 이동할 수 있다

하지만 "찾아보기"가 없는 책도 있는데 이럴 때 어떻게 해야 할까?
결국 책을 첫 페이지부터 열심히 넘기며 확인해보는 방법밖에 없다.

MySQL과의 비유

위의 내용은 MySQL의 인덱스와 비슷한 개념이다.

- 책의 찾아보기 = **MySQL의 인덱스**
- 찾아보기가 있는 책 = **인덱스가 있는 테이블**
- 찾아보기가 없는 책 = **인덱스가 없는 테이블**

비유	MySQL 동작
책의 "찾아보기"	인덱스(B+Tree)
정렬된 단어 목록	정렬된 인덱스 키
단어 옆 페이지 번호	인덱스가 가진 row pointer(레코드 주소)
찾아보기 없이 검색	Full Table Scan
UNIQUE 색인	UNIQUE 인덱스 (중복 불가)

데이터를 찾을 때 인덱스의 사용 여부에 따른 결과값의 차이는 없다. 단지 시간이 오래 걸릴 뿐이다.

1.1 인덱스의 문제점

주의할 점은 인덱스를 제대로 이해하지 못한 채 좋다고 남용하는 것이다.

찾아보기에 만들지 않아도 될 단어들이 쌓이면 쓸데없이 책의 두께만 두꺼워지고, 찾아보기를 사용했는데도 찾는 시간이 찾아보기를 사용하지 않을 때보다 오히려 더 오래 걸릴 수 있다.

실무에서도 운영할 때 비슷한 일이 발생한다.

필요 없는 인덱스를 만드는 바람에 DB가 차지하는 공간만 늘어나고, 인덱스를 이용해 데이터를 찾는 것이 전체 테이블을 찾아보는 것보다 느려진다.

인덱스를 남용했을 때 문제점

1) 쓰기 속도 저하

- 데이터가 변경될 때마다 관련된 모든 인덱스도 갱신해야 한다
- 인덱스가 많을수록 쓰기 작업 비용이 증가한다

예시

```
-- 인덱스가 5개인 테이블에 INSERT 하면
-- 실제 데이터 1건 + 인덱스 5개 업데이트 = 총 6번의 쓰기 작업 발생
INSERT INTO users (id, name, email, phone, address) VALUES (...);
```

2) 디스크 공간 낭비

- 각 인덱스는 별도의 트리 구조를 가지고 있다
- 큰 테이블일수록 저장 공간 비용이 높아진다

예시

원본 테이블: 10GB
인덱스 1개당: 2GB
인덱스 5개: 10GB (원본 테이블만큼의 공간 추가 필요)

3) 캐시 효율 저하

- 인덱스가 많으면 InnoDB 버퍼 풀 같은 캐시에 적재해야 할 페이지가 늘어나 핫 데이터 캐시 효율이 감소한다

동작 원리

버퍼 풀 크기: 1GB
실제 데이터: 500MB
인덱스 데이터: 800MB
→ 인덱스가 버퍼 풀의 대부분을 차지해서 실제 데이터가 캐시에서 밀려남
→ 디스크 I/O 증가

4) 옵티마이저 혼란

- 너무 많은 인덱스 중 어떤 걸 쓸지 판단해야 하는데 비용이 든다

예시

```
-- 10개의 인덱스가 있는 테이블에서
SELECT * FROM users WHERE name = 'Kim' AND age = 30;

-- MySQL 옵티마이저가 고민하는 것들:
-- 1. name 인덱스를 사용할까?
-- 2. age 인덱스를 사용할까?
-- 3. (name, age) 복합 인덱스를 사용할까?
-- 4. Full Table Scan이 더 빠를까?
-- → 이 판단 과정 자체가 오버헤드
```

5) 유지보수 부담

- 스키마 변경 시 인덱스 재생성이 필요하다
- 인덱스 명세를 정확히 이해하고 관리해야 해서 DBA 부담이 증가한다

예시

```
-- 컬럼 타입 변경 시
ALTER TABLE users MODIFY COLUMN email VARCHAR(100);
-- → 해당 컬럼을 포함한 모든 인덱스 재생성 필요

-- 인덱스가 10개면 10개 모두 재생성
-- 대용량 테이블일 경우 수 시간 소요 가능
```

DB에 인덱스를 생성해놓아도 인덱스를 사용하는 것이 빠를지 전체 테이블 검색하는 것이 빠를지 MySQL이 알아서 판단한다. 만약 인덱스를 사용하지 않는다면 사용하지도 않는 찾아보기를 만든 것이므로 쓸데없이 공간을 낭비한 것이다.

1.2 인덱스의 장점과 단점

인덱스는 SELECT에서 즉각적인 효과를 내는 빠른 방법 중 한 가지이다.

즉, 적절한 인덱스를 생성하고 인덱스를 사용하는 SQL을 만든다면 기존보다 아주 빠른 응답 속도를 얻을 수 있다.

장점

- SELECT 문으로 검색하는 속도가 매우 빨라진다
- 그 결과 컴퓨터의 부담이 줄어서 결국 전체 시스템 성능이 향상된다

성능 개선 예시

```
-- 인덱스 없이: 1000만 건 Full Scan → 10초
-- 인덱스 사용: B+Tree 탐색 → 0.01초
SELECT * FROM users WHERE email = 'user@example.com';
```

단점

- 인덱스가 공간을 차지해서 DB 안에 추가적인 공간이 필요하다
 - 테이블 크기의 10-30% 정도 추가 공간 필요
- 처음 인덱스를 만드는 데 시간이 오래 걸릴 수 있다
 - 대용량 테이블의 경우 인덱스 생성에 수 시간 소요 가능
 - 인덱스 생성 중에는 테이블에 락이 걸려 서비스 장애 발생 가능
- SELECT가 아닌 데이터 변경 작업이 자주 일어나면 오히려 성능이 나빠질 수 있다
 - INSERT, UPDATE, DELETE 시 인덱스도 함께 갱신되어야 함
 - 쓰기 작업이 많은 테이블에는 인덱스가 오히려 부담

인덱스를 잘 사용하면 SELECT의 검색 속도가 빨라지지만, 잘못 사용하면 오히려 성능이 나빠진다.

2. 인덱스의 종류

MySQL에서 사용되는 인덱스의 종류는 크게 두 가지로 나뉜다.

클러스터형 인덱스(Clustered Index), **보조 인덱스(Secondary Index)**이다.

비유하자면 클러스터형 인덱스는 영어사전과 같고, 보조 인덱스는 책의 뒤에 찾아보기가 있는 일반적인 책과 같다.

비유로 이해하기

보조 인덱스

- 앞서 설명했던 찾아보기가 별도로 있고, 이를 이용해 실제 찾는 내용이 있는 위치로 이동하는 방식
- 찾아보기(별도 페이지) → 본문 페이지 번호 → 실제 내용

클러스터형 인덱스

- 영어사전처럼 책의 내용이 이미 알파벳 순서대로 정렬되어 있는 방식
- 사전 자체가 이미 정렬되어 있어서 별도의 찾아보기가 필요 없음

클러스터형 인덱스: 인덱스 순서대로 데이터 자체가 정렬

보조 인덱스: 데이터는 따로 있고 색인은 데이터의 위치를 가리킴

2.1 자동으로 생성되는 인덱스

인덱스 는 테이블의 열(컬럼) 단위에 생성되며 하나의 열에는 하나의 인덱스를 생성할 수 있다.

예시 테이블

employees

emp_id	emp_name	dept	age
1	Alice	HR	30
2	Bob	Sales	28
3	Charlie	HR	35
4	David	IT	32
5	Emily	Sales	27

열 하나당 인덱스 하나를 생성하면 테이블에는 우선 5개의 서로 다른 인덱스를 생성할 수 있다.

emp_id를 기본 키로 정의하게 되면 emp_id 열에 클러스터형 인덱스가 생성된다.

그런데 기본 키는 테이블에 하나만 지정할 수 있다고 했다. 결국 클러스터형 인덱스는 테이블에 한 개만 만들 수 있다.

클러스터형 인덱스는 기본 키로 지정하면 자동으로 생성되고 테이블당 1개만 만들 수 있다.

왜 하나만 만들 수 있을까?

테이블의 물리적인 정렬 방식은 한 가지 기준으로만 가능하기 때문이다.

예를 들어 책을 생각해보자:

책 전체를 이름으로 정렬해서 배열할 수 있다.

하지만 같은 책을 나이순으로 동시에 정렬할 수는 없다.

물리적으로 한 가지 기준만 "정렬 상태"가 될 수밖에 없기 때문이다.

→ 그래서 클러스터형 인덱스는 한 테이블에 딱 하나만 존재한다.

기본 키로 생성되는 클러스터형 인덱스

```
CREATE TABLE table1 (  
    col1 INT PRIMARY KEY,  
    col2 INT,  
    col3 INT  
);
```

인덱스 확인하기

테이블의 인덱스를 확인하는 명령어도 존재한다.

SHOW INDEX FROM [테이블명]

여러 컬럼 중에 Key_name, Non_unique, Column_name 을 확인해보자.

결과 분석

- Key_name : PRIMARY라고 적혀 있다 → 기본 키로 설정해서 **자동으로 생성된 인덱스**라는 의미 → 이것이 바로 **클러스터형 인덱스**이다
- Column_name : col1로 설정되어 있다 → col1 열에 인덱스가 만들어져 있다는 의미
- Non_unique : 고유하지 않다는 뜻, 즉 중복이 허용되냐는 뜻
 - 0: False (중복 불가)
 - 1: True (중복 허용)

Non_Unique: 고유하지 않음 = 중복 허용, 0: False, 1: True

고유 키로 생성되는 보조 인덱스

기본 키와 더불어 고유 키로도 인덱스가 자동으로 생성된다.

고유 키로 생성되는 인덱스는 **보조 인덱스**이다.

```
CREATE TABLE table2 (  
    col1 INT PRIMARY KEY,  
    col2 INT UNIQUE,  
    col3 INT UNIQUE  
);
```

```
SHOW INDEX FROM table2;
```

결과 분석

- Key_name : PRIMARY, col2, col3가 적혀 있다
- Column_name : col1, col2, col3가 적혀 있다
- Key_name 이 열 이름이면 보조 인덱스이다

2.2 자동으로 정렬되는 클러스터형 인덱스

클러스터형 인덱스는 기본 키로 지정하면 자동 생성된다는 것을 알았다.
그리고 테이블에 1개만 생성되는 것도 배웠다.

클러스터형의 가장 큰 특징은 **정렬**되어 있다는 것이다.

어떤 열을 기본 키로 지정하면 (클러스터형 인덱스가 생성되면) 그 열을 기준으로 자동 정렬된다.

정렬되지 않은 상태

```
CREATE TABLE member (  
    mem_id CHAR(8),  
    mem_name VARCHAR(10),  
    mem_number INT,  
    addr CHAR(2)  
);
```

이렇게 만들고 데이터를 넣으면 정렬되지 않은 상태로 보여지게 된다.

기본 키 지정 후 자동 정렬

여기서 mem_id를 기본 키로 지정해보자.

```
ALTER TABLE member  
    ADD CONSTRAINT  
    PRIMARY KEY (mem_id);  
  
SELECT * FROM member;
```

이제 클러스터형 인덱스가 생성된 열로 데이터가 자동 정렬이 된다.

동작 원리

정렬 전 :
mem_id: GRL, BLK, APN, MMU
(입력 순서대로 저장)

기본 키 지정 후:

mem_id: APN, BLK, GRL, MMU
(알파벳 순으로 자동 정렬)

2.3 정렬되지 않는 보조 인덱스

고유 키로 지정하면 보조 인덱스가 생성된다고 하였다.
그리고 보조 인덱스는 테이블에 여러 개 설정할 수 있다.

정렬되지 않은 상태

```
CREATE TABLE member (  
    mem_id CHAR(8),  
    mem_name VARCHAR(10),  
    mem_number INT,  
    addr CHAR(2)  
);
```

이 테이블에 데이터를 집어넣고 보아도 정렬되지 않은 상태로 보여진다.

고유 키 지정 후

여기서 mem_id를 고유 키로 설정하고 다시 한번 확인해보자.

```
ALTER TABLE member  
    ADD CONSTRAINT  
    UNIQUE (mem_id);  
  
SELECT * FROM member;
```

보조 인덱스를 생성해도 데이터의 내용이나 순서는 변경되지 않는다.

동작 원리

보조 인덱스 생성 전:

mem_id: GRL, BLK, APN, MMU
(입력 순서대로 저장)

보조 인덱스 생성 후:

실제 데이터: GRL, BLK, APN, MMU (순서 그대로)

인덱스 구조: APN → BLK → GRL → MMU (별도로 정렬되어 관리)

보조 인덱스 사용 시 주의사항

보조 인덱스는 여러 개를 만들 수 있다. 하지만 보조 인덱스를 만들 때마다 데이터베이스의 공간을 차지하게 되고, 전반적으로 시스템에 오히려 나쁜 영향을 미칠 수 있다.

그러므로 꼭 필요한 열에만 적절히 보조 인덱스를 생성하는 것이 좋다.

적절한 보조 인덱스 생성 기준

```
-- 좋은 예 : WHERE 절에 자주 사용되는 컬럼
CREATE INDEX idx_email ON users(email);
-- SELECT * FROM users WHERE email = 'user@example.com';

-- 나쁜 예 : 거의 사용하지 않는 컬럼
CREATE INDEX idx_middle_name ON users(middle_name);
-- 중간 이름으로 검색하는 경우가 거의 없음
```

정리

핵심 개념

- **인덱스**: 데이터를 빠르게 찾기 위한 도구 (책의 찾아보기와 유사)
- **클러스터형 인덱스**: 데이터 자체가 정렬됨 (영어사전과 유사), 테이블당 1개만 가능
- **보조 인덱스**: 별도의 색인 구조 (일반 책의 찾아보기와 유사), 여러 개 생성 가능

인덱스 사용 원칙

- SELECT가 많은 테이블에 적합하다
- INSERT/UPDATE/DELETE가 많은 테이블에는 신중하게 사용한다
- 꼭 필요한 컬럼에만 생성한다
- 인덱스가 실제로 사용되는지 확인한다 (MySQL 옵티마이저 판단)

주의사항

- 인덱스 남용 시 오히려 성능이 저하될 수 있다
- 쓰기 작업 성능 저하, 디스크 공간 낭비, 캐시 효율 저하 등을 고려해야 한다
- 적절한 인덱스 설계와 주기적인 모니터링이 필요하다